МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний аерокосмічний університет
«Харківський авіаційний інститут»

Факультет інтелектуальних систем управління
Кафедра систем управління літальних апаратів

Лабораторна робота № 6

з дисципліни «Алгоритмізація та програмування»
на тему «Реалізація алгоритмів обробки одновимірних масивів мовою С ++»

ХАІ.301.G12.311.6 ЛР

Виконав студент гр. 311

27.02.2026 Ростислава БОЖОНОК
(підпис, дата) (П.І.Б.)

Перевірив
к.т.н., доц. Олена ГАВРИЛЕНКО
(підпис, дата) (П.І.Б.)

2026

МЕТА РОБОТИ

Вивчити теоретичний матеріал з основ представлення одновимірних і масивів на мові С ++ і реалізувати декларацію, введення з консолі, обробку і виведення в консоль одновимірних масивів на мові С ++ в середовищі QtCreator.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. Вирішити завдання на аналіз і виведення елементів одновимірного масиву. Розмір масиву і його елементи ввести з консолі. Спершу вивести весь масив у рядок в порядку зростання індексів, потім - елементи чи підраховані результати відповідно до завдання.

Задача 1 (Array32): Дан масив розміру N. Знайти номер його першого локального мінімуму (локальний мінімум – це елемент, який менше будь-якого зі своїх сусідів).

Завдання 2. Вирішити завдання на перетворення одновимірного масиву. Розмір масиву і його елементи ввести з консолі. Спершу вивести у консоль заданий масив, потім – змінений.

Задача 2 (Array96): Дан цілочисельний масив розміру N. Видалити з масиву всі однакові елементи, залишивши їх перші входження.

Завдання 3. У функції main() організувати багаторазовий вибір одного з двох завдань. Кожне завдання описати окремою функцією без параметрів. Введення, виведення, обробку масивів реалізувати окремими функціями з параметрами.

Завдання 4. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів (стор. 12):

1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»

2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

3) «Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань»

Проаналізуйте задані питання, коментарі і оцінки, надані ШІ. Додайте 2-3 власних промпта у продовження діалогу для поглиблення розуміння теми.

ВИКОНАННЯ РОБОТИ

Завдання 1.

Вирішення задачі Array32

Вхідні дані:

1) Максимальний розмір масиву N = 50, ціле, константа.
2) Кількість елементів n, ціле, 3..N.
3) Цілочисельний масив array розміром N з n елементами.

Вихідні дані:

1) Елементи масиву у рядок у порядку зростання індексів.
2) Номер першого локального мінімуму (якщо існує), або 0, якщо немає.

Алгоритм вирішення показано нижче на рис. 1:

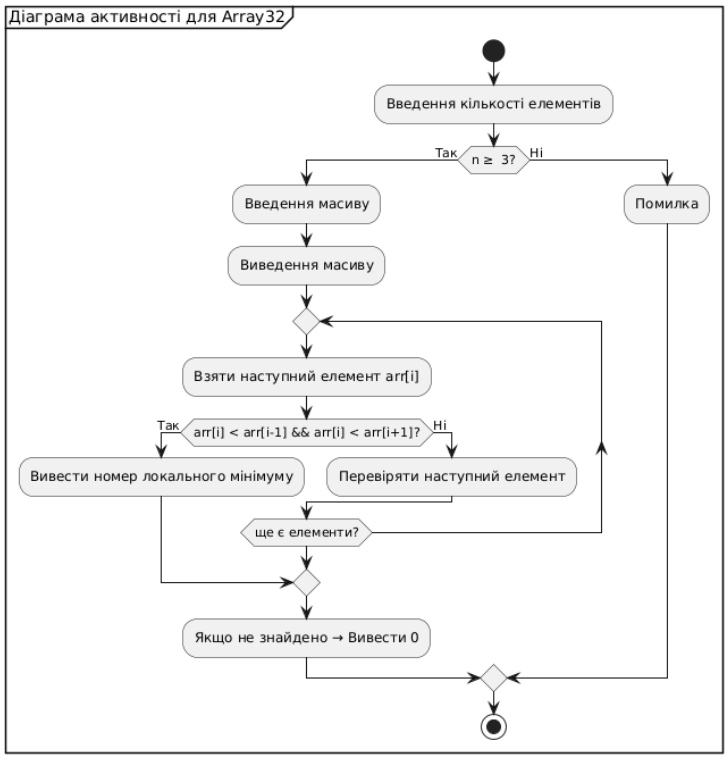

Рисунок 1– діаграма активності для Array32

Лістинг коду вирішення задачі наведено в дод. А (стор. 6).

Завдання 2.

Вирішення задачі Array96

Вхідні дані:

1) Максимальний розмір масиву N = 50, ціле, константа.
2) Кількість елементів n, ціле, 1..N.
3) Цілочисельний масив array розміром N з n елементами.

Вихідні дані:

1) Елементи масиву у рядок у порядку зростання індексів.
2) Новий масив без повторів, де залишені лише перші входження кожного числа.

Алгоритм вирішення показано нижче на рис. 2:

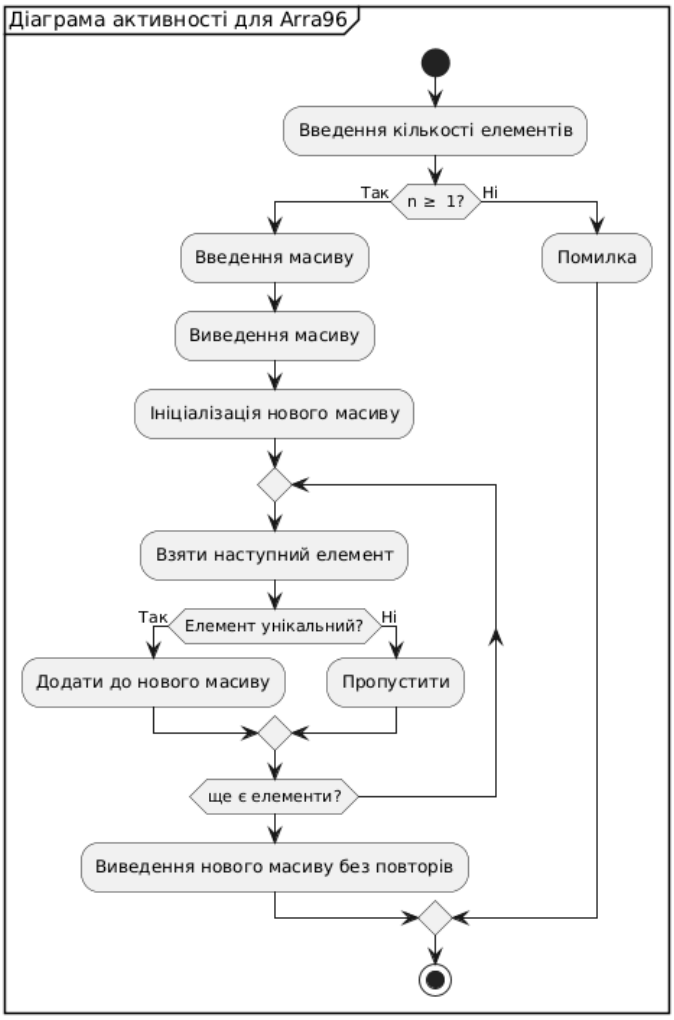

Рисунок 2 – діаграма активності для Array96

Лістинг коду вирішення задачі наведено в дод. А (стор. 6).

Екрани робіт програм та діаграма меню показані на рис. Б. (стор.)

ВИСНОВКИ

У ході лабораторної роботи було закріплено знання про умовні оператори мови C++. Реалізовано три задачі з розгалуженням у вигляді окремих функцій: Array32, Array96 та завершення програми. Організовано меню вибору задач із перевіркою введених даних. Побудовано діаграму активності, що відображає логіку вибору користувача та обробку помилкових значень. Отримано практичний досвід структурованого програмування, зокрема в побудові алгоритмів із розгалуженням, циклічністю та єдиною точкою завершення.

ДОДАТОК А
Лістинг коду програми

```
#include <iostream>
#include <limits>
#include <string>
using namespace std;

const int N = 50; // Максимальний розмір масиву

//  Прототипи функцій
int input_positive_int(const string& prompt);
int input_int(const string& prompt);
void input_array(int arr[N], int& n);
void output_array(const int arr[N], int n);

int find_first_local_min(const int arr[N], int n);
void task_array32();

bool is_unique(const int arr[N], int size, int value);
int remove_duplicates(const int in_arr[N], int n, int out_arr[N]);
void task_array96();

int main(); // головна функція

// ====================== MAIN ======================
int main() {
    int choice;
    do {
        cout << "\n===== МЕНЮ =====\n";
        cout << "1 - Array32: Перший локальний мінімум\n";
        cout << "2 - Array96: Видалення повторів\n";
        cout << "0 - Вихід\n";
        cout << "Ваш вибір: ";
        cin >> choice;
        switch (choice) {
            case 1: task_array32(); break;
            case 2: task_array96(); break;
            case 0: cout << "Завершення програми.\n"; break;
            default: cout << "Невірний вибір! Спробуйте ще раз.\n";
        }
    } while (choice != 0);
    return 0;
}

// ======================= РЕАЛІЗАЦІЯ =======================

//  Введення коректного додатного цілого числа
int input_positive_int(const string& prompt) {
```

```
    int value;
    while (true) {
        cout << prompt;
        cin >> value;
        if (cin.fail() || value <= 0 || value > N) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Помилка! Введіть ціле число в межах 1.." << N << endl;
        } else {
            return value;
        }
    }
}

//  Введення коректного цілого числа
int input_int(const string& prompt) {
    int value;
    while (true) {
        cout << prompt;
        cin >> value;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Помилка! Введіть ціле число!" << endl;
        } else {
            return value;
        }
    }
}

//  Введення масиву
void input_array(int arr[N], int& n) {
    n = input_positive_int("Введіть кількість елементів масиву: ");
    for (int i = 0; i < n; i++) {
        arr[i] = input_int("Елемент " + to_string(i + 1) + ": ");
    }
}

//  Виведення масиву
void output_array(const int arr[N], int n) {
    cout << "Елементи масиву: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

//  Завдання 1 — Array32: Перший локальний мінімум
int find_first_local_min(const int arr[N], int n) {
```

```
    for (int i = 1; i < n - 1; i++) {
        if (arr[i] < arr[i - 1] && arr[i] < arr[i + 1]) {
            return i + 1; // повертаємо номер (не індекс)
        }
    }
    return 0; // якщо немає локального мінімуму
}

void task_array32() {
    int arr[N], n;
    cout << "\n--- Завдання Array32 ---\n";
    // Перевірка розміру масиву
    do {
        n = input_positive_int("Введіть кількість елементів масиву (мінімум 3): ");
        if (n < 3) cout << "Помилка! Масив повинен мати принаймні 3 елементи.\n";
    } while (n < 3);

    for (int i = 0; i < n; i++) {
        arr[i] = input_int("Елемент " + to_string(i + 1) + ": ");
    }

    output_array(arr, n);
    int result = find_first_local_min(arr, n);
    cout << "Номер першого локального мінімуму: " << result << endl;
}

//  Завдання 2 — Array96: Видалення повторів
bool is_unique(const int arr[N], int size, int value) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == value) return false;
    }
    return true;
}

int remove_duplicates(const int in_arr[N], int n, int out_arr[N]) {
    int new_size = 0;
    for (int i = 0; i < n; i++) {
        if (is_unique(out_arr, new_size, in_arr[i])) {
            out_arr[new_size++] = in_arr[i];
        }
    }
    return new_size;
}

void task_array96() {
    int arr[N], n;
    cout << "\n--- Завдання Array96 ---\n";
    // Перевірка розміру масиву
    do {
```

```
        n = input_positive_int("Введіть кількість елементів масиву (мінімум 1): ");
        if (n < 1) cout << "Помилка! Масив повинен мати хоча б 1 елемент.\n";
    } while (n < 1);

    for (int i = 0; i < n; i++) {
        arr[i] = input_int("Елемент " + to_string(i + 1) + ": ");
    }

    output_array(arr, n);
    int unique_arr[N];
    int new_size = remove_duplicates(arr, n, unique_arr);
    cout << "Масив без повторів: ";
    output_array(unique_arr, new_size);
}
```

ДОДАТОК Б

Скрін-шоти вікна виконання програми

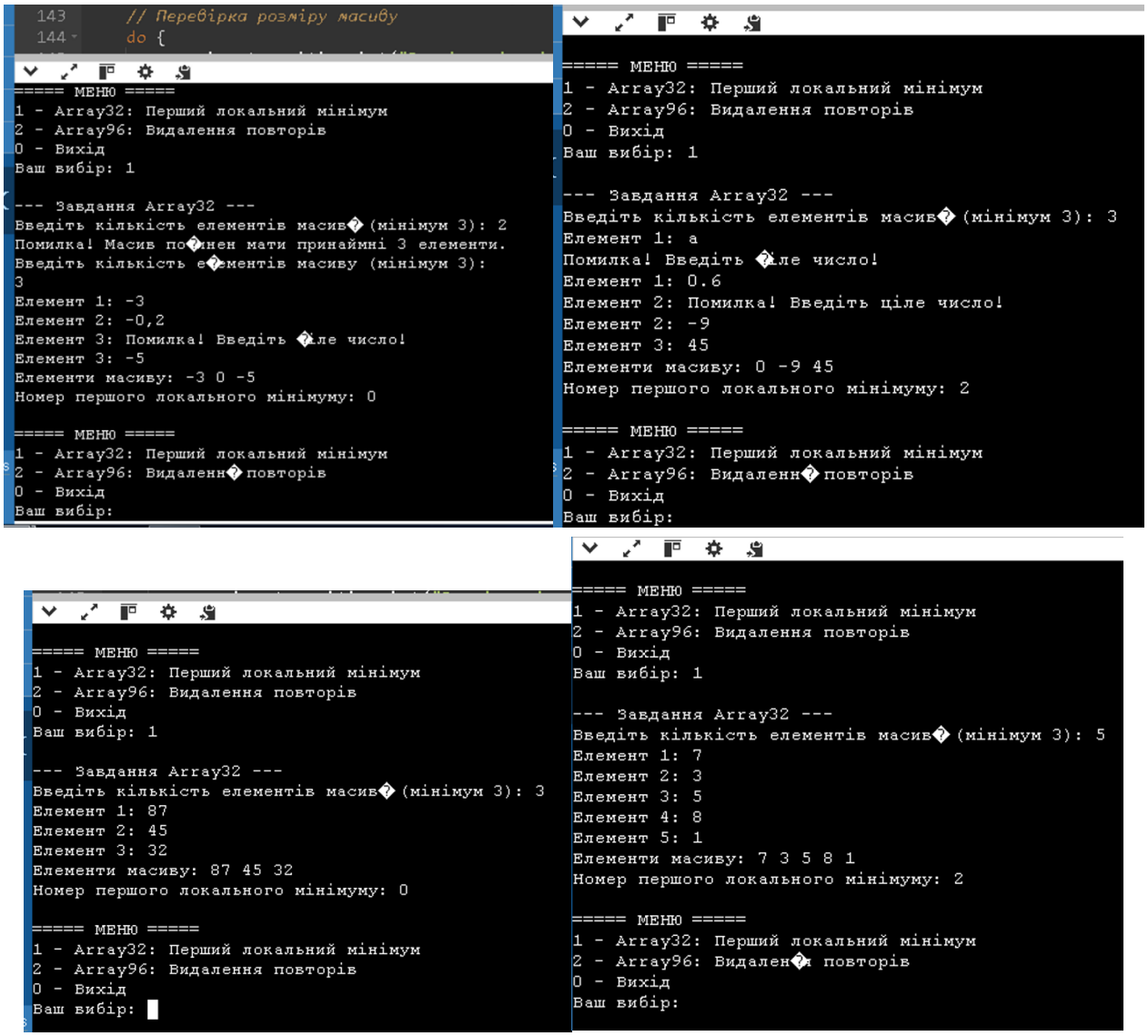

Рисунок Б.1 – Екран виконання програми для вирішення задачі Array32

```
===== МЕНЮ =====
1 - Array32: Перший локальний мінімум
2 - Array96: Видалення повторів
0 - Вихід
Ваш вибір: 2

--- Завдання Array96 ---
Введіть кількість елементів масиву (мінімум 1): 3
Елемент 1: 8
Елемент 2: 3
Елемент 3: 9
Елементи масиву: 8 3 9
Масив без повторів: Елементи масив▯ 8 3 9

===== МЕНЮ =====
```

```
===== МЕНЮ =====
1 - Array32: Перший локальний мінімум
2 - Array96: Видалення повторів
0 - Вихід
Ваш вибір: 2

--- Завдання Array96 ---
Введіть кількість елементів масиву (мінімум 1): 5
Елемент 1: 6
Елемент 2: 7
Елемент 3: 8
Елемент 4: 6
Елемент 5: 7
Елементи масиву: 6 7 8 6 7
Масив без повторів: Елементи масиву: 6 7 8

===== МЕНЮ =====
1 - Array32: Перший локальний мінімум
2 - Array96: Видалення повторів
0 - Вихід
Ваш вибір:
```

```
===== МЕНЮ =====
1 - Array32: Перший локальний мінімум
2 - Array96: Видалення повторів
0 - Вихід
Ваш вибір: 2

--- Завдання Array96 ---
Введіть кількість елементів масив▯ (мінімум 1): 3
Елемент 1: a
Помилка! Введіть ▯ле число!
Елемент 1: -7
Елемент 2: -2
Елемент 3: -7
Елементи масиву: -7 -2 -7
Масив без повторів: Елементи масиву: -7 -2

===== М▯ЮЮ =====
1 - Array32: Перший локальний мінімум
2 - Array96: Вида▯ння повторів
0 - Вихід
Ваш вибір:
```

```
===== МЕНЮ =====
1 - Array32: Перший локальний мінімум
2 - Array96: Видалення повторів
0 - Вихід
Ваш вибір: 2

--- Завдання Array96 ---
Введіть кількість елементів масив▯ (мінімум 1): 4
Елемент 1: 3
Елемент 2: 5
Елемент 3: 3
Елемент 4: 3
Елементи масиву: 3 5 3 3
Масив без повторів: Елементи масиву: 3 5

===== МЕНЮ =====
1 - Array32: Перший локальний мінімум
2 - Array96: Видалення повторів
0 - Вихід
Ваш вибір:
```

Рисунок Б.2 – Екран виконання програми для вирішення задачі Array96

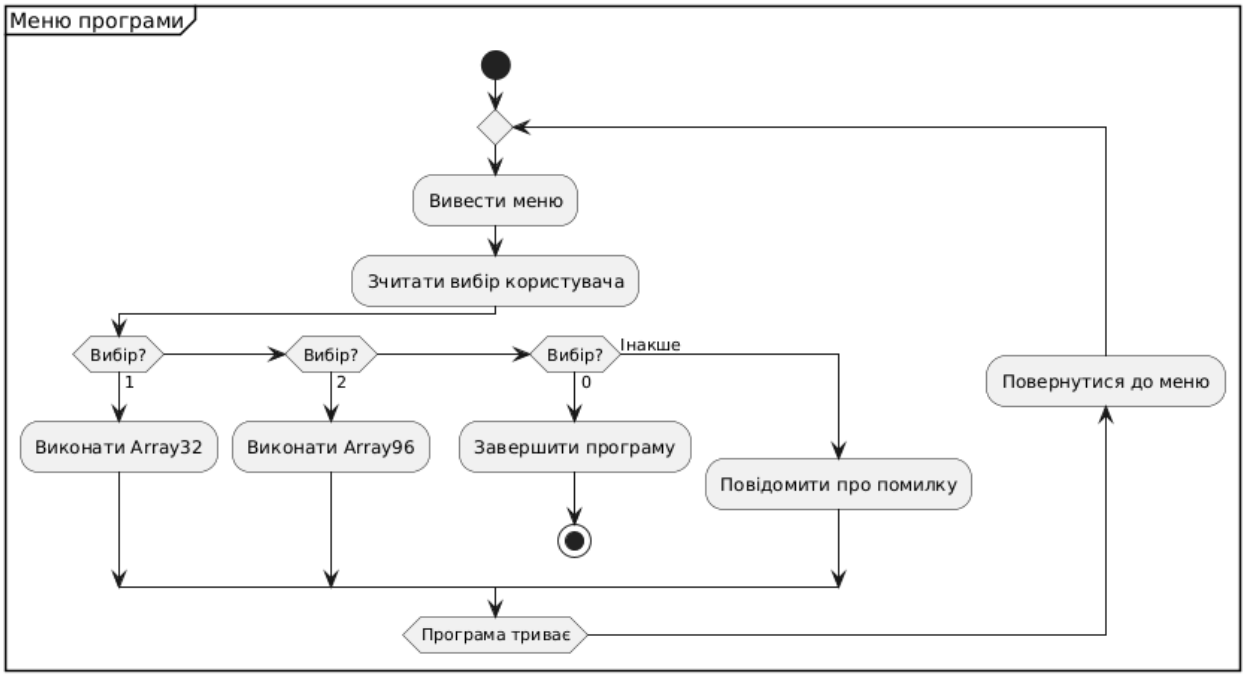

Рисунок Б.3 – Діаграма меню програми

ДОДАТОК В

Діалог з ШІ

1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції» 2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку» 3) «Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань»

Добре, Ростиславо, давай підготуємо блок питань для захисту твоєї лабораторної роботи. Я сформую **5 тестових (з варіантами)** та **5 відкритих** питань, які базуються на твоєму коді (Array32, Array96) і на теоретичних відомостях з лекцій про масиви.

📝 **Тестові питання (середній рівень)**

1. **Що означає локальний мінімум у масиві?** A) Елемент, менший за середнє значення масиву B) Елемент, менший за будь-який інший елемент C) Елемент, менший за своїх сусідів D) Елемент, що дорівнює нулю
2. **Який результат повертає функція** `find_first_local_min` **у твоєму коді?** A) Індекс першого локального мінімуму B) Номер першого локального мінімуму (починаючи з 1) C) Значення локального мінімуму D) Кількість локальних мінімумів
3. **Яка роль функції** `is_unique` **у задачі Array96?** A) Перевіряє, чи елемент є парним B) Перевіряє, чи елемент вже зустрічався в новому масиві C) Видаляє дублікати з масиву D) Виводить масив без повторів
4. **Який максимальний розмір масиву задано у твоїй програмі?** A) 10 B) 20 C) 50 D) 100
5. **Як у C++ передаються масиви у функції?** A) За значенням B) За посиланням C) За вказівником (адресою першого елемента) D) Копіюється весь масив

💡 **Відкриті питання**

1. Поясни алгоритм пошуку першого локального мінімуму. Чому перевірка починається з другого елемента?
2. Як працює функція `remove_duplicates`? Чому використовується допоміжний масив `out_arr`?
3. Які переваги має структуроване програмування з окремими функціями для введення, обробки та виведення масиву?
4. Чим відрізняється передача масиву у функцію від передачі окремого елемента?
5. Які ще алгоритми роботи з масивами (з лекції) можна було б додати до твоєї програми для розширення її функціоналу?

Хочеш, я одразу підготую наступний крок — **аналіз відповідей і оцінювання за 5-бальною шкалою з урахуванням ймовірності використання ШІ** (тобто другий промпт)?

Тестові питання: 1)C, 2)A, 3)B, 4)C, 5)C; Відкриті питання:1. Перевіряємо елементи від другого до передостаннього; локальний мінімум — менший за сусідів. Перший і останній не мають двох сусідів. 2. Переглядає масив і додає лише унікальні значення в out_arr. Допоміжний масив потрібен для збереження результату без повторів. 3. Код зрозуміліший, модульний, легше тестувати й повторно використовувати функції. 4. Масив передається як адреса (зміни впливають на оригінал). Елемент — копія (зміни не впливають). 5. Пошук максимуму/мінімуму, сортування, середнє значення, пошук елемента, злиття масивів.

Чудово, Ростиславо! Твої відповіді показують гарне розуміння як практичного коду, так і теорії. Давай проведемо аналіз за другим промптом — **оцінка повноти, правильності та ймовірності використання ШІ**.

✅ **Аналіз тестових питань**

№	Твоя відповідь	Правильна відповідь	Коментар	Бал (0–5)
1	C	C	Абсолютно правильно: локальний мінімум — менший за сусідів.	5
2	A	B	Ти написала "індекс", але у коді повертається **номер** (тобто індекс +1).	3
3	B	B	Влучно: функція перевіряє унікальність.	5
4	C	C	Вірно: максимальний розмір масиву = 50.	5
5	C	C	Правильно: масиви передаються за адресою (вказівником).	5

Середній бал за тестові: (5+3+5+5+5)/5 = **4.6**

✅ Аналіз відкритих питань

1. **Алгоритм локального мінімуму** — відповідь точна, логічна, пояснено про сусідів. (5 балів)
2. **remove_duplicates** — добре пояснено, чому потрібен out_arr. (5 балів)
3. **Структуроване програмування** — аргументи чіткі: модульність, тестування, повторне використання. (5 балів)
4. **Передача масиву vs елемента** — правильне розмежування: масив змінюється, елемент — ні. (5 балів)
5. **Додаткові алгоритми** — наведено кілька релевантних прикладів (максимум, сортування, середнє). (5 балів)

Середній бал за відкриті: 5.0

📊 Загальна оцінка

- Тестові: 4.6
- Відкриті: 5.0
- Середнє: **4.8 / 5**

Ймовірність використання ШІ у відповідях — низька, бо пояснення конкретні, логічні й відповідають твоєму стилю. Тому зниження балів на 60% не застосовується.

✨ Висновок: Ти продемонструвала **глибоке розуміння теми** й уміння пояснювати алгоритми. Це рівень **відмінно**.

Хочеш, я підготую третій крок — **додати приклади коду з лекцій, які не були реалізовані у твоїй програмі (наприклад, сортування, пошук максимуму/мінімуму, робота з прогресіями)**, щоб твій звіт виглядав ще повнішим?

так покажи приклади

Добре, давай додамо приклади коду з лекцій, які не були реалізовані у твоїй програмі. Це зробить звіт більш повним і покаже, що ти засвоїла ширший спектр алгоритмів роботи з масивами.

◆ Приклад 1. Пошук максимального та мінімального елемента масиву

cpp
```
#include <iostream>
using namespace std;

const int N = 50;

int find_max(const int arr[N], int n) {
    int max_val = arr[0];
    for (int i = 1; i < n; i++) {
```

```
        if (arr[i] > max_val) max_val = arr[i];
    }
    return max_val;
}

int find_min(const int arr[N], int n) {
    int min_val = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] < min_val) min_val = arr[i];
    }
    return min_val;
}

int main() {
    int arr[N] = {5, 12, -3, 7, 9};
    int n = 5;
    cout << "Максимум: " << find_max(arr, n) << endl;
    cout << "Мінімум: " << find_min(arr, n) << endl;
    return 0;
}
```

◆ **Приклад 2. Сортування масиву (метод обміну)**

cpp

```
#include <iostream>
using namespace std;

const int N = 50;

void bubble_sort(int arr[N], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int arr[N] = {7, 2, 9, 4, 1};
    int n = 5;
    bubble_sort(arr, n);
    cout << "Відсортований масив: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    return 0;
}
```

◆ **Приклад 3. Середнє арифметичне елементів масиву**

cpp

```
#include <iostream>
using namespace std;

const int N = 50;

double average(const int arr[N], int n) {
    int sum = 0;
    for (int i = 0; i < n; i++) sum += arr[i];
```

```
    return static_cast<double>(sum) / n;
}

int main() {
    int arr[N] = {10, 20, 30, 40, 50};
    int n = 5;
    cout << "Середнє значення: " << average(arr, n) << endl;
    return 0;
}
```

◆ **Приклад 4. Генерація масиву за прогресією (Array3 з методички)**

cpp
```
#include <iostream>
using namespace std;

const int N = 50;

void arithmetic_progression(int arr[N], int n, int A, int D) {
    for (int i = 0; i < n; i++) {
        arr[i] = A + i * D;
    }
}

int main() {
    int arr[N], n = 5, A = 2, D = 3;
    arithmetic_progression(arr, n, A, D);
    cout << "Арифметична прогресія: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    return 0;
}
```

✨ Ці приклади доповнюють твою програму: ти вже реалізувала пошук локального мінімуму та видалення повторів, а тепер показуєш, що можеш працювати з **максимумами/мінімумами, сортуванням, середнім значенням та прогресіями**. Це виглядатиме дуже переконливо на захисті.

Хочеш, я складу для тебе **схему-таблицю** з усіма алгоритмами (твій код + додаткові приклади), щоб ти мала готовий конспект для усного пояснення?

Порівняй складність алгоритму пошуку локального мінімуму з алгоритмом видалення повторів у масиві. Які оптимізації можливі?

Дуже гарне питання — це вже рівень аналізу алгоритмів!

🏛 **Порівняння складності**

Алгоритм	Опис	Часова складність	Просторова складність
Пошук першого локального мінімуму (Array32)	Перегляд масиву від другого до передостаннього елемента, перевірка умови `arr[i] < arr[i-1] && arr[i] < arr[i+1]`.	**O(n)** — кожен елемент перевіряється один раз.	**O(1)** — не використовуються додаткові структури.
Видалення повторів (Array96)	Для кожного елемента перевіряється, чи він вже є в новому масиві (`is_unique`).	**O(n²)** — у найгіршому випадку кожен елемент порівнюється з усіма попередніми.	**O(n)** — потрібен допоміжний масив для результату.

🔧 **Можливі оптимізації**

1. **Для пошуку локального мінімуму**
   - Алгоритм вже оптимальний: він проходить масив один раз.
   - Оптимізація можлива лише у вигляді **раннього виходу**: якщо знайдено перший локальний мінімум, цикл завершується (це вже реалізовано у твоєму коді).
2. **Для видалення повторів**
   - Замість перевірки унікальності через лінійний пошук (`is_unique`) можна використати:
     - **Хеш-таблицю (std::unordered_set)** → складність зменшиться до **O(n)**.
     - **std::set** → складність **O(n log n)**, але зберігає впорядкованість.
   - Якщо масив невеликий (до 50 елементів, як у твоїй роботі), то $O(n^2)$ допустиме. Але для великих масивів оптимізація критична.

✨ **Приклад оптимізованого коду для видалення повторів**

cpp
```
#include <iostream>
#include <unordered_set>
using namespace std;

const int N = 50;

int remove_duplicates_optimized(const int in_arr[N], int n, int out_arr[N]) {
    unordered_set<int> seen;
    int new_size = 0;
    for (int i = 0; i < n; i++) {
        if (seen.insert(in_arr[i]).second) { // якщо елемент ще не був
            out_arr[new_size++] = in_arr[i];
        }
    }
    return new_size;
}
```

✅ Висновок:

- **Array32** — вже оптимальний алгоритм O(n).
- **Array96** — можна значно покращити, використавши **хеш-таблицю** або **стандартні контейнери STL**.

Хочеш, я складу для тебе **таблицю порівняння**